

2026-05-06-p1-f18-no-flicker-rendering-design

Phase 1 / Feature 18 — No-Flicker Rendering

Date: 2026-05-06 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

1. Goal

Ship a real, end-to-end **no-flicker terminal renderer** for the HelixCode CLI agent so that streaming LLM tokens and tool result blocks update *in place* without redrawing the whole viewport, and degrade *gracefully* to plain line-by-line output when stdout is not a TTY (CI logs, pipes, redirected output, dumb terminals).

Three concrete user surfaces ship together:

1. **render package** (`helix_code/internal/render/`) — a small custom ANSI/carriage-return renderer (claude-code's approach; Q1=A). NO new external dependencies. Uses `\r\033[K` for in-place line updates, ANSI cursor positioning (`\033[<n>A / \033[<n>B`) for multi-line blocks, and a double-buffered viewport for tool-result rendering. Two implementations behind a single `Renderer` interface: `ansiRenderer` (TTY mode) and `plainRenderer` (non-TTY fallback). A `RendererFactory` selects between them at startup based on the env var + `term.IsTerminal(int(os.Stdout.Fd()))`.
2. **Streaming LLM tokens + tool result blocks instrumentation** (Q2=B) — the agent loop's existing `provider.GenerateStream` callback feeds tokens into the renderer's `WriteToken(text)` so a multi-token reply appears as ONE in-place updating line per logical line, NOT one new printed line per token. Tool result blocks (LSP diagnostics from F13, sandbox stdout from F14, subagent results from F15, smart-edit diffs from F17) are rendered as `Frame` (slice of lines) via `Renderer.RenderFrame(frame)`; if a previous frame for the same block id exists, dirty-region diff (Q3=B) emits *only* the changed lines via cursor-positioning + clear-line. Other CLI output (start-up banners, / command outputs, `helixcode` cobra command output, build-time logs) remains line-based as today — it does not flow through the renderer.
3. **Env-var configuration** (Q5=B) — `HELIXCODE_RENDER=plain|fancy|auto` (default `auto`). `auto = fancy` when stdout is a TTY, `plain` otherwise. NO slash command. NO cobra subcommand. The variable is read once at process startup and snapshotted into the `RenderMode` selected by the factory.

The non-TTY graceful degrade (Q4=B) is structurally enforced: `plainRenderer` MUST emit zero ANSI escape sequences. Output to a non-TTY writer (a pipe, a CI log capture, `>file`, a dumb terminal) MUST be byte-identical to what `fmt.Fprintln` would produce — complete, in order, line-terminated, and free of `\r` carriage-returns that would corrupt log capture or grep pipelines.

The dirty-region diff (Q3=B) is the explicit anti-bluff lever for “no flicker is real, not just claimed”: the renderer tracks a `Viewport` of the previous frame's lines, computes the line-by-

line LCS-trivial dirty set on each new frame, and emits ANSI sequences ONLY for the lines that changed. A frame with one changed line of a five-line block emits exactly one `\033[<n>\r\033[K<line>\033[<n>B`-style sequence — not five rewrites disguised as a fancy update.

Out of scope for v1: full screen-mode TUI (alternative-screen-buffer `\033[?1049h` is deferred to F18.5); `tcell/termbox` adoption in the CLI hot path (the `applications/terminal_ui/` `Fyne+tvview` TUI is a separate code path and not changed by F18); SIGWINCH terminal-resize responsiveness (the renderer reads width once at startup; resize-during-render is documented as “next frame picks up new width on next factory init or process restart”); mouse input; bracketed paste; truecolor / 24-bit color negotiation (we emit only the standard 8/16-color SGR set when needed for emphasis); Windows-pre-10 ANSI shimming (we rely on Go 1.16+’s automatic VT-mode enablement on Windows 10+).

Anti-bluff: a `render` package whose code exists but is never wired into the agent’s streaming hot path (so the agent still goes through `fmt.Print`), OR a `plainRenderer` that claims fallback but still emits a `\r` that breaks `grep-on-redirected-output`, OR a `dirty-region diff` that returns the same byte sequence whether one line changed or all five — each is a critical defect (§5.2). The single largest bluff vector for F18 is “renderer initialised but never called from production”, which compiles, passes naive unit tests, and silently degrades the user experience to today’s flicker-prone output without anyone noticing.

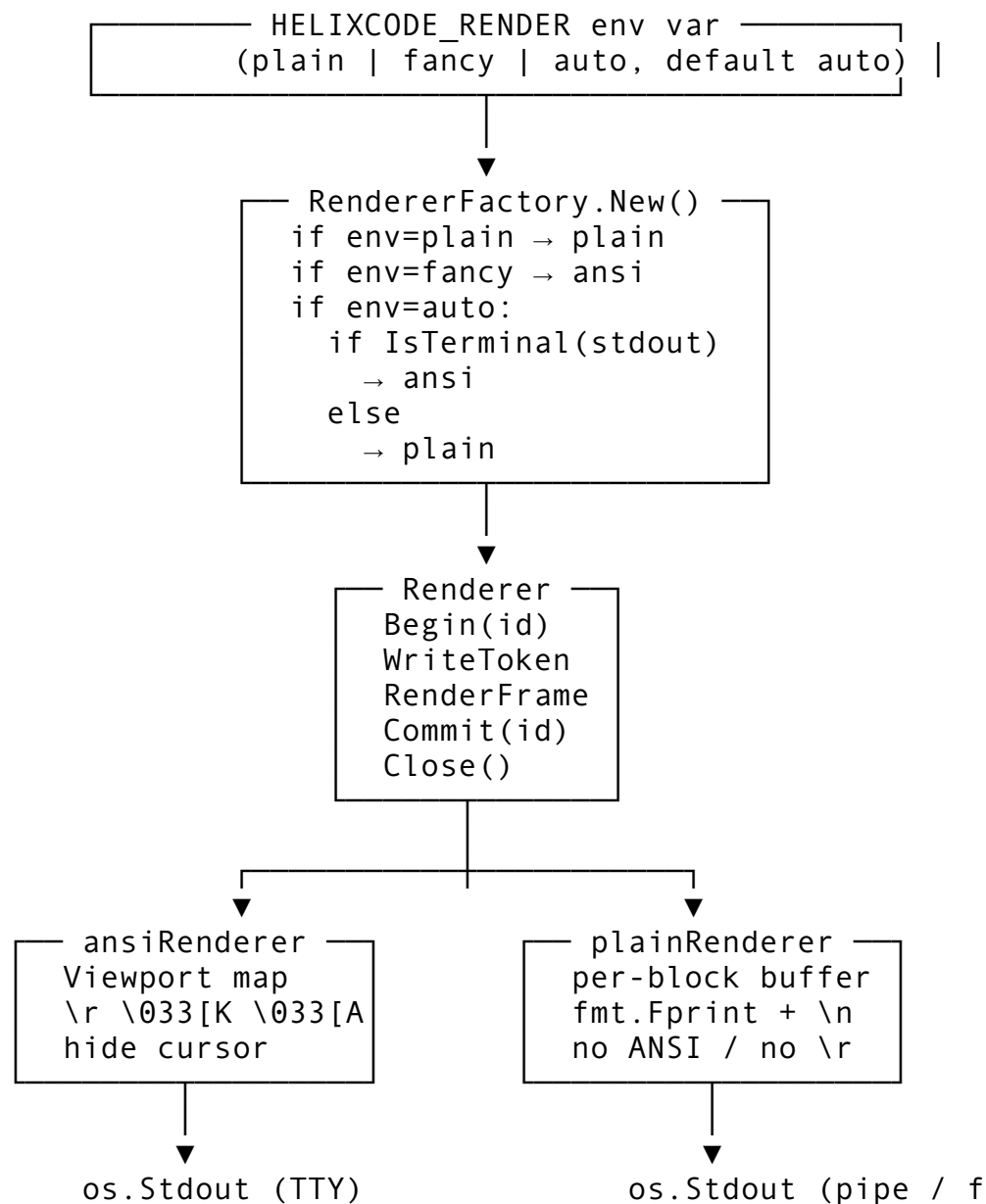
2. Architecture

Three layers, all under `helix_code/internal/render/`, plus thin wiring into the existing CLI streaming path and tool-result printing path:

- **Renderer interface** (`types.go`) — five methods: `Begin(blockID string)` (mark a new in-progress streaming/frame block, returns the block’s frame handle), `WriteToken(blockID string, text string)` (append streaming text into the current line of the block; the renderer batches and emits in-place updates), `RenderFrame(blockID string, frame Frame)` (replace the block’s content with the given frame; dirty-region diff against the previous frame for that block emits only changed lines), `Commit(blockID string)` (finalise the block — emits a trailing `\n` for `ansiRenderer` so the cursor leaves the in-place region; no-op for `plainRenderer`), `Close() error` (renderer-wide cleanup; restores cursor visibility on `ansiRenderer`). The renderer is **single-writer** — the agent loop drives it; concurrent calls from goroutines are serialised by an internal mutex but the spec does NOT promise interleaved-stream rendering (see §5.3).
- **ansiRenderer** (`ansi_renderer.go`) — TTY-mode impl. Owns an `io.Writer` (typically `os.Stdout`) + a `map[string]*Viewport` keyed by block id. `WriteToken` appends to the active line of the block’s viewport, then emits `\r\033[K<currentLine>` to redraw THAT line in place. `RenderFrame` runs `Viewport.Diff(newFrame)` to compute the dirty line set, then for each dirty line emits `\033[<n>\r\033[K<line>\033[<n>B` (move up `n`, carriage-return, clear-line, write, move down `n`). On `Commit`, emits `\n` to push the cursor past the block. Hides the cursor (`\033[?25l`) at `Begin`, restores it (`\033[?25h`) at `Close`.
- **plainRenderer** (`plain_renderer.go`) — non-TTY fallback. Owns an `io.Writer`. `WriteToken` accumulates tokens into a per-block string buffer; on the first newline character (or on `Commit`), flushes the accumulated buffer with a trailing `\n` via a single `fmt.Fprint`. `RenderFrame` prints each line followed by `\n`. `Commit` flushes any unflushed token buffer. **Zero ANSI escape sequences. Zero `\r` characters. Zero**

cursor positioning. A non-TTY writer sees only printable characters and `\n`. This is enforced by a unit test that scans the captured byte buffer for any byte in the set `{0x1b, 0x0d}` and fails if any are present.

- `Viewport` (`viewport.go`) — Frame buffer + dirty-line tracking. `Viewport{lines []string; lastLines []string}.Diff(newFrame Frame) []int` returns the indices of changed lines (`lastLines[i] != newFrame.Lines[i]`, plus any added or removed indices). Pure Go, no dependencies.
- `Frame` (`types.go`) — type `Frame struct { ID string; Lines []string }`. A read-only structural slice the renderer renders or diffs against the viewport.
- `RendererFactory` (`factory.go`) — selects an impl at process startup. Reads `HELIXCODE_RENDER` (default `auto`); when `auto`, calls `term.IsTerminal(int(os.Stdout.Fd()))`. Returns a configured `Renderer` plus the chosen `RenderMode` (an enum: `RenderModePlain`, `RenderModeFancy`, surfaced for logging / test assertions). The factory is the *only* place the env var and the TTY probe are read; the rest of the codebase only sees the `Renderer` interface.



Wire points (existing code; one new line each):

- **LLM streaming hook:** `cmd/cli/main.go::handleGenerate` already streams via `provider.GenerateStream(ctx, req, chunkChan)` (lines 1080–1090). F18 replaces the current `for chunk := range chunkChan { fmt.Printf("%s", chunk.Content) }` with `r.Begin(id); for chunk := range chunkChan { r.WriteToken(id, chunk.Content) }; r.Commit(id)`.
- **Agent loop streaming hook** (when streaming is wired in the loop): `internal/agent/base_agent.go::executeTaskWithLLM` currently calls `Generate` only (line 408). F18 does NOT change that path in v1 — the agent loop’s non-streaming `Generate` prints the response in one shot via `processLLMResponse`, which is line-based and stays as-is. The renderer’s `WriteToken` path is exercised by the CLI’s `handleGenerate` (where streaming is opt-in via `--stream`) and by future agent-loop streaming work (out of scope for F18).
- **Tool result frame rendering hook:** `internal/tools/registry.go::Execute` returns `interface{} results`; the CLI prints them via `fmt.Println` today. F18 adds a thin helper `RenderToolResult(r Renderer, id string, result interface{})` (in `render/tool_helpers.go`) that converts known result types (LSP diagnostics, sandbox stdout, smart-edit diffs, subagent results) into a `Frame` and calls `r.RenderFrame(id, frame); r.Commit(id)`. Unknown result types fall back to `fmt.Println(result)`. This helper is invoked from `cmd/cli/main.go` at the spots that already print tool output; the registry itself stays unchanged.

Why a custom ANSI renderer (Q1=A) and not `tcell / termbox / bubbletea`: - **Zero new external deps** is a hard programme rule for F18 (matches F17’s anti-drift discipline). - The render surface is two narrow modes (in-place token streaming + small-block frame rendering) — far less than what a full TUI library brings. We don’t need event loops, focus management, or alt-screen for F18. - The full-screen TUI in `applications/terminal_ui/` already uses `tview/tcell`. Adding `bubbletea` here would create a third terminal-rendering paradigm in the codebase. The CLI is a streaming line-oriented tool; a tiny purpose-built renderer matches the surface.

Why `env-var` only (Q5=B) and not `slash + cobra`: - Render mode is a *startup* property — the renderer is constructed once and the `env` value is snapshotted into the active mode. Switching mid-process would require renderer re-initialisation, viewport reset, and unwinding any in-flight in-place updates. v1 punts on this complexity. - A `slash` command would be useful for in-process toggling (deferred to F18.5 if user demand emerges); a `cobra` subcommand `helixcode render -mode` would just print the resolved mode (cosmetic; users can `echo $HELIXCODE_RENDER` for that).

3. Components

3.1 New files

- `helix_code/internal/render/types.go` — `Renderer` interface, `Frame` struct, `RenderMode` enum (`RenderModePlain`, `RenderModeFancy`), error sentinels.
- `helix_code/internal/render/types_test.go`.
- `helix_code/internal/render/ansi_renderer.go` — `ansiRenderer`; in-place line update via `\r\033[K`; multi-line frame rendering via `\033[<n>A + \r\033[K + \033[<n>B`; cursor hide/show via `\033[?25l / \033[?25h`.

- `helix_code/internal/render/ansi_renderer_test.go` — `bytes.Buffer` capture; assert exact byte sequences for known `WriteToken / RenderFrame` inputs.
- `helix_code/internal/render/plain_renderer.go` — line-by-line `fmt.Fprint` fallback. Zero ANSI. Zero `\r`.
- `helix_code/internal/render/plain_renderer_test.go` — assert captured buffer contains NO `0x1b` and NO `0x0d` bytes for any input.
- `helix_code/internal/render/viewport.go` — `Viewport{lines, lastLines} + Diff(newFrame Frame) []int`. Pure Go.
- `helix_code/internal/render/viewport_test.go`.
- `helix_code/internal/render/factory.go` — `NewRenderer(opts Options) (Renderer, RenderMode)`; reads `env + isatty`; constructor-injection of `writer + IsTTY` for tests.
- `helix_code/internal/render/factory_test.go` — fake `writer + IsTTY=true / IsTTY=false`; assert correct impl chosen for every (`env`, `isatty`) combination.
- `helix_code/internal/render/tool_helpers.go` — `RenderToolResult(r Renderer, id string, result interface{})` glue for known tool-result types.
- `helix_code/internal/render/tool_helpers_test.go`.
- `helix_code/tests/integration/render_test.go` — `//go:build integration`. Real fake-TTY (a `bytes.Buffer + IsTTY=true` injected via constructor option) and real non-TTY (`bytes.Buffer` only). Exercises full pipeline.
- `helix_code/tests/integration/cmd/p1f18_challenge/main.go` — runtime evidence harness.
- `challenges/p1-f18-no-flicker-rendering/CHALLENGE.md + run.sh`.

3.2 Modified files

- `helix_code/cmd/cli/main.go` — three blocks: (1) construct the renderer at startup via `render.NewRenderer(...)`; (2) replace the streaming-print loop in `handleGenerate` (lines 1080–1090) with `Begin/WriteToken/Commit`; (3) wrap tool-result printing through `render.RenderToolResult`.
- `helix_code/cmd/cli/main.go` — register a `defer renderer.Close()` so the cursor is restored if the process exits mid-render.

No new external dependencies (§3.5).

3.3 Types

```
// internal/render/types.go
```

```
type RenderMode int
```

```
const (
    RenderModePlain RenderMode = iota // line-based; no ANSI;
                                     non-TTY fallback
    RenderModeFancy                  // ANSI in-place updates +
                                     dirty-region diff
)
```

```
func (m RenderMode) String() string
```

```

// Frame is a structural snapshot of a rendered block; the
//     renderer diffs
// against the previous Frame for the same blockID and emits only
//     changed
// lines.
type Frame struct {
    ID      string
    Lines []string
}

// Renderer is the unified surface the CLI uses for streaming
//     token output
// and tool-result frame rendering. Both impls satisfy this
//     interface; the
// factory picks the right one at startup.
//
// Concurrency: single-writer. Internal mutex serialises calls;
//     the spec
// does NOT promise interleaved rendering of two distinct
//     streaming blocks
// at once (see §5.3).
type Renderer interface {
    Begin(blockID string)
    WriteToken(blockID, text string)
    RenderFrame(blockID string, frame Frame)
    Commit(blockID string)
    Close() error
    Mode() RenderMode
}

// Error sentinels.
var (
    ErrUnknownBlockID = errors.New("render: unknown blockID
        (Begin not called)")
    ErrInvalidEnvMode =
        errors.New("render: HELIXCODE_RENDER must be one of
            plain|fancy|auto")
)

// Constants – env var + valid values.
const (
    EnvRenderMode = "HELIXCODE_RENDER"
    ModePlainEnv  = "plain"
    ModeFancyEnv  = "fancy"
    ModeAutoEnv   = "auto"
)

```

```

// internal/render/factory.go

type Options struct {
    Writer io.Writer // defaults to os.Stdout
    IsTTY  *bool           // nil = probe via
                    // term.IsTerminal(stdout.Fd()); true/false = forced (test
                    // seam)
    Env    func(string)
          string // nil = os.Getenv; injectable for tests
    Width
          int // 0 = probe via term.GetSize(stdout.Fd());
          else forced
}

// NewRenderer constructs a Renderer per the resolved (env,
// isatty) policy.
// Returns (renderer, mode, err). err is non-nil only when env
// var is set
// to an invalid value.
func NewRenderer(opts Options) (Renderer, RenderMode, error)

// internal/render/ansi_renderer.go

type ansiRenderer struct {
    w      io.Writer
    width  int
    mu     sync.Mutex

    // per-block in-progress state
    blocks map[string]*blockState
}

type blockState struct {
    viewport      *Viewport
    activeLine    string // accumulating-token buffer for
                    streaming (token mode)
    activeLineIdx
          int // for multi-line blocks: which line is being
                    streamed into
    inProgress   bool // Begin called, Commit not yet
}

func newAnsiRenderer(w io.Writer, width int) *ansiRenderer

// implements Renderer

// internal/render/plain_renderer.go

type plainRenderer struct {

```

```

    w io.Writer
    mu sync.Mutex

    // per-block accumulating buffer (so streaming tokens flush
    // as whole
    // lines, never as partial-line writes that break grep)
    buffers map[string]*strings.Builder
}

func newPlainRenderer(w io.Writer) *plainRenderer

// implements Renderer

// internal/render/viewport.go

type Viewport struct {
    lines []string
    lastLines []string // previous frame; nil before first
    RenderFrame
}

func NewViewport() *Viewport

// Diff returns the 0-indexed line indices that changed between
// lastLines
// and newFrame.Lines. Includes indices for added lines (i ≥
// len(lastLines))
// and removed lines (i ≥ len(newFrame.Lines)). Pure; does not
// mutate.
func (v *Viewport) Diff(newFrame Frame) []int

// Apply commits newFrame.Lines as the new lastLines. Called by
// the
// renderer AFTER it has emitted the dirty-line writes.
func (v *Viewport) Apply(newFrame Frame)

// internal/render/tool_helpers.go

// RenderToolResult inspects known tool-result types and renders
// them as
// a Frame via r.RenderFrame; unknown types fall back to
// fmt.Fprintln on
// the renderer's underlying writer (or os.Stdout for plain non-
// buffer).
//
// Currently recognised result types (extension point):
// - lsp.DiagnosticReport (F13)
// - sandbox.ExecutionResult (F14)
// - smartedit.SmartEditResult (F17)

```



```
// - subagent.Result (F15)
//
// For unrecognised types, the helper prints a single line via
//   fmt.Fprintln
// (delegated through the renderer's plain mode for non-TTY
//   safety).
func RenderToolResult(r Renderer, blockID string, result
    interface{})
```

3.4 User surfaces

Env-var only (Q5=B). The renderer mode is selected by `HELIXCODE_RENDER`:

Value	Behaviour
plain	Always use <code>plainRenderer</code> ; no ANSI; no <code>\r</code> ; line-by-line <code>fmt.Fprintln</code> .
fancy	Always use <code>ansiRenderer</code> ; in-place updates; cursor positioning; dirty-region diff. WARNING: fancy on a non-TTY writer (e.g., piped output) WILL emit raw ANSI sequences; the user opted in.
auto (default)	<code>term.IsTerminal(stdout.Fd())</code> : TTY → fancy; non-TTY → plain.
(unset)	Treated as auto.
(any other value)	<code>NewRenderer</code> returns <code>ErrInvalidEnvMode</code> ; <code>cmd/cli/main.go</code> logs the error and falls back to plain (so the agent NEVER fails to start because of a typo'd env var).

No slash command (Q5=B). **No cobra subcommand** (Q5=B). The user can: - `HELIXCODE_RENDER=plain helixcode --prompt ... --stream` to force plain in a TTY (e.g., for screen-recording tools that mishandle in-place updates). - `HELIXCODE_RENDER=fancy helixcode ... | tee log` to keep ANSI in the captured log (rare; usually undesirable). - Default (unset) is the right answer for 99% of users.

The chosen mode is logged at INFO at startup (`render mode: fancy (auto-resolved, stdout is TTY)`). The full ANSI byte stream is NEVER logged at any level (would re-emit the very escapes we just produced into the log file, defeating the point).

3.5 New external dependencies

None. F18 is built entirely on the Go standard library (`io`, `os`, `bytes`, `strings`, `fmt`, `sync`, `errors`) plus `golang.org/x/term` (already in `go.mod` of the inner module, currently as `// indirect` at v0.41.0 — confirmed via `grep "golang.org/x/term" helix_code/go.sum`). After T02 introduces a direct import of `golang.org/x/term`, `go mod tidy` will promote the line in `go.mod` from `// indirect` to a direct `require` — that is

the one expected `go.mod` change for F18, and **no new entries in** `go.sum` (it's already pinned at v0.41.0). Brief justification:

- ANSI control is a small finite set of byte sequences — pure constants + `fmt.Sprintf` for the parameterised ones.
- Dirty-region diff is a `for i := range max(len(old), len(new))` line-comparison loop.
- TTY detection is a single `term.IsTerminal(int)` call from `golang.org/x/term`.
- Width probe (one-shot, at factory time) is `term.GetSize(int)`; not strictly needed for v1 (the renderer doesn't wrap; long lines are passed through and the terminal does its own wrap), but included so SIGWINCH-deferred work in F18.5 has a place to read.

`go mod tidy` after T02 must produce **no new entries in** `go.sum` (only the indirect→direct promotion in `go.mod`). T10's verification step asserts this.

3.6 Existing-code constraints

- `cmd/cli/main.go::handleGenerate` (lines 1050–1102) — the only existing site that actually streams from a provider is here, gated by `--stream`. F18 modifies lines 1080–1090. The non-streaming path (`provider.Generate + single fmt.Println(resp.Content)`) stays as-is — `Generate` returns one whole response, which is line-based by nature and benefits zero from in-place updates.
- `cmd/cli/main.go` startup (around the existing logger / cobra setup) — F18 inserts `renderer, mode, err := render.NewRenderer(render.Options{})` before any tool/slash registration. `defer renderer.Close()` is registered immediately after.
- `internal/agent/base_agent.go::executeTaskWithLLM` (lines 360–426) — currently calls `Generate` (non-streaming). F18 does NOT change this; agent-loop streaming is a separate, larger change (out of scope; the renderer is built so a future patch can wire it in via a single `WriteToken` call site without re-architecting).
- `internal/tools/registry.go::Execute` (line 250+) — F18 does NOT modify the registry. Tool result printing is at the *call sites* of `Execute` in `cmd/cli/main.go`; `RenderToolResult` is invoked there.
- `applications/terminal_ui/` (Fyne / tvview) — F18 is the **CLI** renderer; the desktop GUI and the tvview TUI are separate code paths that already use their own rendering. They are NOT touched by F18.
- `golang.org/x/term` is at v0.41.0 in `go.sum` (verified) — already transitively required by other `golang.org/x` deps; F18 promotes it to a direct require.

4. Data flow

4.1 Process startup → renderer construction

```
main()
├─ logger init
├─ renderer, mode, err := render.NewRenderer(render.Options{})
│   └─ env := os.Getenv("HELIXCODE_RENDER")
│       └─ switch env {
│           case "plain": → plainRenderer
│           case "fancy": → ansiRenderer
│           case "auto", "":
│               if term.IsTerminal(int(os.Stdout.Fd())) → ansiRenderer
```

```

    }
    else → plainRenderer
    default: return ErrInvalidEnvMode
}
width = term.GetSize(stdout.Fd()) (best-effort; 80 fallback)
if err != nil: logger.Warn(...); fallback to plain
logger.Info("render mode: %s", mode)
defer renderer.Close()
... rest of bootstrap (tools, slash, cobra) ...
run command

```

4.2 LLM token streaming → in-place line update

```

handleGenerate(ctx, prompt, model, ..., stream=true):
├─ provider := c.llmProvider
├─ chunkChan := make(chan llm.LLMResponse, 100)
├─ id := "llm-" + shortHash(prompt + model + ts)
├─ r.Begin(id)
├─ go provider.GenerateStream(ctx, req, chunkChan) // existing call site
├─ for chunk := range chunkChan:
│   r.WriteToken(id, chunk.Content)
│   ┌─ ansiRenderer:
│   │   mu.Lock; defer mu.Unlock
│   │   bs := blocks[id]
│   │   bs.activeLine += chunk.Content
│   │   // detect newline boundary
│   │   if strings.ContainsRune(bs.activeLine, '\n'):
│   │       lines := strings.Split(bs.activeLine, "\n")
│   │       for i := 0; i < len(lines)-1; i++:
│   │           // commit completed line: \r\033[K + line + \n
│   │           fmt.Fprintf(w, "\r\x1b[K%s\n", lines[i])
│   │           bs.activeLine = lines[len(lines)-1] // remaining part in buffer
│   │       // emit current partial line in-place
│   │       fmt.Fprintf(w, "\r\x1b[K%s", bs.activeLine)
│   └─ plainRenderer:
│       mu.Lock; defer mu.Unlock
│       buf := buffers[id]
│       buf.WriteString(chunk.Content)
│       // flush whole lines
│       for buf.String() contains '\n':
│           idx := index of first '\n'
│           fmt.Fprintln(w, buf.String()[0:idx])
│           buf.Reset(); buf.WriteString(buf.String()[idx+1:])
│       // partial line stays in buffer until newline or Commit
├─ r.Commit(id)
│   ┌─ ansiRenderer: emit final '\n' so cursor leaves the in-place region
│   └─ plainRenderer: flush remaining buffer with trailing '\n'
└─ done

```

4.3 Tool result frame rendering → dirty-region diff

```

RenderToolResult(r, id, result):
├─ frame := convertResultToFrame(result)
└─ // type-switch over known result types; default → single-line Frame

```

```

├─ r.Begin(id)
├─ r.RenderFrame(id, frame)
│   └─ ansiRenderer:
│       mu.Lock; defer mu.Unlock
│       bs := blocks[id]
│       dirty := bs.viewport.Diff(frame)
│       if first frame (lastLines == nil):
│           // emit all lines, leave cursor at start of last line
│           for i, line := range frame.Lines:
│               fmt.Fprintf(w, "%s\n", line)
│       else:
│           // dirty-region diff: for each changed line, move up the
│           // amount, clear, rewrite; restore cursor position
│           for _, idx := range dirty:
│               up := len(bs.viewport.lastLines) - idx // distance
│               fmt.Fprintf(w, "\x1b[%dA\r\x1b[K%s\x1b[%dB\r", up, f
│       bs.viewport.Apply(frame)
│   └─ plainRenderer:
│       mu.Lock; defer mu.Unlock
│       for _, line := range frame.Lines:
│           fmt.Fprintln(w, line)
├─ r.Commit(id)
└─ done

```

4.4 Mode-resolution truth table

HELIXCODE_RENDER	stdout is TTY	Selected impl	Mode logged
(unset)	yes	ansiRenderer	fancy (auto)
(unset)	no	plainRenderer	plain (auto)
auto	yes	ansiRenderer	fancy (auto)
auto	no	plainRenderer	plain (auto)
plain	yes	plainRenderer	plain (forced)
plain	no	plainRenderer	plain (forced)
fancy	yes	ansiRenderer	fancy (forced)
fancy	no	ansiRenderer	fancy (forced — user opt-in)
garbage	*	plainRenderer (after warn-log)	plain (fallback after invalid env)

4.5 Token-buffer flush boundaries

Both renderers respect newline-aligned flushing for **logging-safety**:

- `ansiRenderer.WriteToken` flushes the partial line in-place after every token (the user sees streaming progress) AND emits a real `\n` at every newline boundary in the token stream (so the line is committed to scrollbar and the next line starts fresh).

- `plainRenderer.WriteToken` accumulates into a per-block buffer and flushes ONLY at newline boundaries (so a piped consumer never sees a half-line). On `Commit`, any remaining partial buffer is flushed with a trailing `\n`.

This means `plainRenderer` output is byte-identical to “the LLM emitted whole lines and the agent printed each with `fmt.Println`” — the user never observes “streaming” in plain mode (correct for non-TTY) but the FINAL output is complete and well-formed.

5. Error handling, edge cases, and anti-bluff

5.1 Error paths

- **Invalid env var** — `NewRenderer` returns `ErrInvalidEnvMode`; `cmd/cli/main.go` logs a warning and constructs a `plainRenderer` directly so the agent never fails to start.
- **Begin not called before WriteToken / RenderFrame** — the renderer auto-creates the block state lazily (no `ErrUnknownBlockID` on first write; this matches the streaming reality where the first token *is* the block start). `Commit` for an unknown block id is a no-op.
- **Stream emits zero tokens** — `Begin` followed by `Commit` with no `WriteToken` between. `ansiRenderer.Commit` emits a single `\n` (so the cursor is on a fresh line). `plainRenderer.Commit` emits nothing.
- **Token stream contains ANSI escapes** — the LLM may emit color codes in its output. v1 policy (§5.4): pass-through — write them verbatim. The terminal interprets them; non-TTY consumers see them as noise. Sanitization is deferred to F18.5.
- **Token contains carriage return \r** — `ansiRenderer` writes `\r` verbatim (the terminal handles it). `plainRenderer` STRIPS `\r` from the buffer before flushing (CRITICAL: a `\r` in non-TTY output corrupts log capture — `grep` against a log file with `\r`-overwrites returns mangled lines). The strip is silent and documented.
- **Frame with zero lines** — treated as “clear the block” — for `ansiRenderer`, emit `\033[<n>A\r\033[K\033[<n>B` for each previously-rendered line; for `plainRenderer`, emit nothing.
- **Close while blocks are in progress** — flush all open blocks (commit each), then emit `\033[?25h` (show cursor) for `ansiRenderer`. `plainRenderer.Close` flushes any pending buffers.
- **SIGINT / process kill mid-render** — `defer renderer.Close()` in `main` runs as long as the process exits cleanly; on `SIGKILL` the cursor may stay hidden. Documented; F18.5 will install a `SIGINT` handler.
- **Concurrent calls from multiple goroutines** — internal mutex serialises; the renderer is single-writer (§5.3). Two simultaneous `Begin/WriteToken/Commit` cycles for *different* blockIDs interleave the writes by mutex order; the visible result is “block A committed, then block B committed” — NOT “A and B updating in place at the same time.” This is intentional.

5.2 Anti-bluff (CONST-035 / §11.9) — LOUD

The single largest bluff vector for F18 is “renderer code exists but is never called from the production hot path.” This compiles, passes naive unit tests, and silently leaves the agent on today’s flicker-prone `fmt.Print` path. Common bluff variants:

1. **(a) Renderer initialised but never invoked from streaming hot path** — `render.NewRenderer` is called in `main()` and the returned `Renderer` is stored on the CLI struct, but the streaming loop in `handleGenerate` still calls

- `fmt.Printf("%s ", chunk.Content)`. **Defence:** an integration test runs `handleGenerate` against a fake `LLMProvider` that emits 10 tokens, captures stdout via a `bytes.Buffer` writer injected through the renderer's `Options.Writer`, and asserts the captured bytes match the `WriteToken` byte signature (specifically: contains `\r\033[K` between consecutive tokens in fancy mode, OR contains exactly 10 `\n`-terminated complete-line writes in plain mode — NOT 10 separate `fmt.Print` calls each appending a token).
2. **(b) Test asserts ANSI sequences are in some buffer but never observes the actual production-path render** — a unit test that constructs an `ansiRenderer` directly, writes to it, and checks the buffer is fine, but does NOT prove the agent's code path actually goes through that renderer. **Defence:** a Challenge phase ("STREAMING") wires up an actual CLI instance with a fake `LLMProvider` and a `bytes.Buffer` injected as `os.Stdout`-substitute; runs the streaming code path end-to-end; asserts the capture contains the renderer's signature byte sequences AND does NOT contain any unwrapped `fmt.Printf("%s ", chunk.Content)` artefact (specifically: there is NO sequence of tokens separated by single spaces that matches today's bluffy implementation).
 3. **(c) "Plain mode" claims fallback but emits a \r that breaks log capture** — the plain renderer accidentally calls `fmt.Fprintf(w, "\r%s\n", line)` (so the captured log has `\r\n` line terminators that mangle `grep`-against-the-log). **Defence:** a unit test for `plainRenderer` writes a 100-token stream + a 5-line frame and scans the captured byte buffer for ANY occurrence of the bytes `0x1b` (ANSI ESC) OR `0x0d` (CR); any presence FAILS the test loudly.
 4. **(d) Dirty-region diff claims to skip unchanged lines but actually re-emits everything** — `Viewport.Diff` returns all indices regardless of whether they changed, OR the renderer ignores the diff result and just re-renders the whole frame. **Defence:** a unit test renders a 5-line frame, then renders the same frame with line 3 changed (lines 0,1,2,4 identical), captures the second-render byte sequence, and asserts: (i) the captured bytes contain EXACTLY ONE `\033[<n>A` cursor-up sequence and EXACTLY ONE `\r\033[K` clear-line sequence; (ii) the captured bytes' length is $< (5 \times \text{longest-line-bytes})$ — i.e., the renderer demonstrably did NOT re-emit all 5 lines. A second test asserts that when ALL 5 lines change, the captured output DOES contain 5 dirty-line emissions. Both directions are pinned.

Required real-execution criteria (these define what "no-flicker rendering works" means in F18):

1. **Unit tests** — render to a `bytes.Buffer` via constructor injection; assert exact byte sequences for the ANSI controls. For example, `TestAnsiRenderer_WriteToken_EmitsClearLineBetweenTokens` writes `["hello", "world"]`, captures the buffer, and asserts the bytes match the regex `^\r\x1b\[Khello\r\x1b\[Khelloworld$` (after token-2 lands; note token-2 includes both characters because `WriteToken` appends to `activeLine`, so the line evolves `hello` → `helloworld` and each version is emitted via clear-line + rewrite).
2. **Integration tests** (`-tags=integration`) — exercise the renderer against a fake "TTY" (a `bytes.Buffer` with `IsTTY=true` injected via `Options.IsTTY = boolPtr(true)`) and a non-TTY (just `bytes.Buffer`, `IsTTY=false`). Verify both paths. Includes: (i) factory mode-resolution truth table (all 9 rows); (ii) full streaming pipeline through a fake LLM provider; (iii) tool-result frame rendering with sample LSP-diagnostic / smart-edit-diff inputs.

3. Challenge harness — exercises:

- **Streaming LLM tokens phase (always runs)**: capture rendered output for 10 streamed tokens against a fake LLM provider; assert in fancy mode the capture contains 10 `\r\033[K` clear-line sequences (one per token's in-place update) AND $\leq 1 \backslash n$ (the final commit), NOT 10 separate `\n`-terminated lines. Assert in plain mode the capture contains exactly the concatenated token text (no ANSI; no `\r`).
- **Tool block render phase (always runs)**: render a 3-line LSP diagnostic block; render again with line 2 changed; assert in fancy mode the second render's byte count is strictly less than the first render's byte count AND the second render contains exactly ONE cursor-up + one clear-line + one cursor-down (dirty-region proof via captured byte length). Assert in plain mode both renders are 3 line-terminated writes.
- **TTY-fallback phase (always runs)**: with non-TTY writer (a `bytes.Buffer` with `IsTTY=false`), render the streaming + tool-block pipelines; scan the captured bytes for ANY `0x1b` or `0x0d` byte; FAIL if any are present.
- **Real TTY phase (gated on `os.Stdout` actually being a TTY)**: detect TTY via `term.IsTerminal(int(os.Stdout.Fd()))`; if non-TTY, SKIP-OK with marker SKIP-OK: P1-F18 real-TTY phase requires interactive terminal (CI / pipe). When TTY, run the streaming pipeline against `os.Stdout` for real, then immediately Close the renderer and assert the renderer's `Mode()` is `RenderModeFancy`. NOTE: this phase is observational — it can't capture the actual rendered visual; the assertion is mode + completion-without-error + no-panic. The fake-TTY phases above provide the byte-level evidence.

Document the gating loudly. The Challenge MUST run all three always-run phases regardless of TTY availability (CI compatibility).

4. **Challenge MUST exit non-zero on any byte-signature mismatch.** “renderer ran without error” is NEVER acceptable. The Challenge asserts positive byte-evidence (e.g., “the captured stdout contains the literal bytes `\r\0x1b[K`”) for every phase.

Concrete forbidden phrases (anti-bluff smoke):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" \
internal/render && echo BLUFF || echo clean
```

Must always print `clean`.

CONST-042 secret-content protection (mandatory):

LLM streaming output may contain secrets the user has prompted the model to discuss (rare but possible — e.g., the model echoes back an API key from the user's prompt). The mechanism:

- **No rendered-frame text at INFO level.** The host process logs `INFO render mode: fancy (auto-resolved, stdout is TTY)` at startup ONLY. Per-frame and per-token output is NEVER logged at any level — there is no `logger.Info("rendered token: %s", text)` call site anywhere in the renderer. A unit test scans the renderer's source files for `logger.Info(*token / logger.Info(*frame / logger.Debug(*token.*\".*\")` and FAILS if any match. (Yes — even DEBUG; the renderer doesn't log content. Period.)
- **Rendered output to stdout IS visible** — the user invoked the agent with a streaming prompt; they get to see the tokens. This is by design. Terminal output is local-only by

default; if the user is recording / pasting their terminal that's their decision, not the renderer's.

- **No rendered text in the Challenge's saved evidence file** — the Challenge harness records `BYTE LENGTHS` and `SIGNATURE PRESENCE` (e.g., “captured 247 bytes, contains `\r\x1b[K`, contains 10 clear-line sequences”), NOT the rendered text itself. The harness's `stdout` (which captures the actual rendered output for human inspection) is treated as transient and not committed.

5.3 Concurrent writes — single-writer assumption

The renderer is **single-writer**. The agent loop is the writer. Tool execution that returns a result and then triggers a frame render is serialised through the agent loop's main goroutine. The internal `sync.Mutex` exists only to serialise calls that happen to come in concurrently (e.g., a tool callback fires a render-frame while the streaming loop is mid-token); it does NOT promise interleaved rendering of two distinct in-progress streaming blocks at once.

Documented limitation: if two goroutines call `Begin` for two different block IDs and start interleaving `WriteToken` calls, the visible output will be jumbled (mutex order, not stream-aware merging). This is an acceptable v1 tradeoff because the agent loop is structured around one-stream-at-a-time. F18.5 may add a “stream multiplexer” if user demand emerges.

5.4 Embedded ANSI in tool output — pass-through

Tool results may contain ANSI escapes (e.g., `git diff` output with `--color=always` from a sandboxed shell command). v1 policy: **pass-through verbatim**. The renderer treats incoming text as opaque bytes. In fancy mode the terminal interprets them (intended). In plain mode the bytes are flushed to the writer as-is (so a user piping plain-mode output to a file sees the raw escapes; this is a known caveat).

Sanitization (stripping ANSI from tool output before plain-mode rendering) is deferred to F18.5 because the right policy is non-obvious — some users WANT the colors preserved in their captured logs (terminal-recasting tools render them); others WANT them stripped (grep-friendliness). v1 picks the simpler default; v2 may add a `HELIXCODE_RENDER_STRIP_ANSI=1` toggle.

5.5 Terminal width too small — no special handling

v1 does NOT wrap or truncate long lines. If a line exceeds the terminal width: - **Fancy mode**: the terminal performs its own soft-wrap. The next `\r\033[K` clear-line will clear only the *current* terminal line, leaving the wrapped continuation lines as artefacts on screen. This is a known limitation; users with very narrow terminals will see leftover text. - **Plain mode**: the line is written verbatim with a trailing `\n`. The terminal soft-wraps as usual. No artefacts because no in-place updates.

Wrapping/truncation logic is deferred to F18.5. The renderer's `Width` field (set at factory time via `term.GetSize`) is captured but unused in v1; F18.5 will use it.

5.6 SIGWINCH terminal-resize handling — deferred

v1 reads terminal width once at factory time. A resize during the process lifetime is NOT detected. The next process invocation picks up the new width. F18.5 will install a `SIGWINCH` handler.

6. Testing

6.1 Unit (real bytes . Buffer for capture; fake IsTTY via constructor option; no mocks of stdlib)

Types (types_test.go): - TestRenderMode_String_Plain / TestRenderMode_String_Fancy. - TestErrorSentinels_DistinctErrorsIs. - TestEnvVarConstants_ExactValues.

Viewport (viewport_test.go): - TestViewport_Diff_FirstFrame_AllNew. - TestViewport_Diff_NoChange_EmptyDirty. - TestViewport_Diff_OneLineChanged_OneIndex. - TestViewport_Diff_AllLinesChanged_AllIndices. - TestViewport_Diff_NewFrameLonger_AddedIndicesIncluded. - TestViewport_Diff_NewFrameShorter_RemovedIndicesIncluded. - TestViewport_Apply_UpdatesLastLines.

ansiRenderer (ansi_renderer_test.go): - TestAnsiRenderer_WriteToken_EmitsClearLineSequence — write "hi", assert capture is \r\x1b[Khi. - TestAnsiRenderer_WriteToken_TwoTokens_TwoClearLines — write ["hello", "world"], assert capture matches expected exact byte sequence. - TestAnsiRenderer_WriteToken_NewlineCommitsLine_NextStartsAtCol0. - TestAnsiRenderer_RenderFrame_FirstFrame_EmitsAllLines. - TestAnsiRenderer_RenderFrame_OneLineChanged_EmitsOneCursorUpClearWrite_Dir - TestAnsiRenderer_RenderFrame_AllLinesChanged_EmitsAllDirtyEmissions. - TestAnsiRenderer_Commit_EmitsTrailingNewline. - TestAnsiRenderer_BeginCloseSequence_HidesAndShowsCursor. - TestAnsiRenderer_TokenContainsAnsiEscape_PassedThroughVerbatim (§5.4). - TestAnsiRenderer_ConcurrentBlocksSerialisedByMutex (two goroutines; assert both completed; no panic).

plainRenderer (plain_renderer_test.go): - TestPlainRenderer_WriteToken_PartialLineBufferedUntilNewline — write ["hello"] (no newline), assert capture is empty. - TestPlainRenderer_WriteToken_FullLineFlushed. - TestPlainRenderer_RenderFrame_PrintsLines. - TestPlainRenderer_NoAnsiEscapesEver — write 100 tokens + 5 frames; assert capture contains zero 0x1b bytes. - TestPlainRenderer_NoCarriageReturnsEver — same; assert zero 0x0d bytes. - TestPlainRenderer_TokenContainsCR_StrippedSilently — write "hello\rworld"; assert capture contains helloworld (no CR). - TestPlainRenderer_TokenContainsAnsi_PassedThroughVerbatim — note that pass-through here defers to §5.4; the test pins the v1 behaviour. (CONTRAST: NoAnsiEscapesEver checks the renderer's OWN emissions; this test checks that user-provided text is not stripped.) - TestPlainRenderer_Commit_FlushesPendingBuffer.

Factory (factory_test.go): - All 9 rows of the truth table in §4.4 — TestFactory_AutoTTY_Fancy / TestFactory_AutoNonTTY_Plain / TestFactory_PlainTTY_Plain / TestFactory_FancyNonTTY_Fancy_UserOptIn /

TestFactory_Garbage_FallbackPlain_WarnLogged / etc. -
TestFactory_Mode_LoggedAtStartup_NeverLogsContent (CONST-042). -
TestFactory_DefaultOptions_UsesOsStdout. -
TestFactory_ConstructorInjectionOverridesEnv (test seam).

Tool helpers (tool_helpers_test.go): -
TestRenderToolResult_LSPDiagnostic_ConvertedToFrame. -
TestRenderToolResult_SandboxResult_ConvertedToFrame. -
TestRenderToolResult_SmartEditResult_ConvertedToFrame. -
TestRenderToolResult_UnknownType_FallsBackToFprintln.

Anti-bluff source-scan (in factory_test.go or a dedicated file): -
TestNoLoggerInfoOrDebugTakesContent_CONST042 — `grep -r 'logger\\.\\(Info\\|Debug\\).*\\(token\\|frame\\|content\\|text\\)' internal/render/* . go` MUST return zero matches. The test runs the `grep` at test time against the renderer source files; fails on any match.

6.2 Integration (//go:build integration)

tests/integration/render_test.go (ALWAYS-runs; no infrastructure dep):

- `TestRender_StreamingPipeline_FancyMode_EmitsClearLineSequences` — fake LLM provider emits 10 tokens; capture via `bytes.Buffer`; assert capture contains exactly 10 `\r\x1b[K` sequences.
- `TestRender_StreamingPipeline_PlainMode_NoAnsiNoCR` — same fake provider; capture under plain mode; assert capture contains zero `0x1b` and zero `0x0d`; assert capture's printable text concatenation matches the concatenated tokens.
- `TestRender_ToolResultFrame_DirtyDiffEmitsOneLineUpdate` — render a 5-line frame, then render with line-2 changed; assert second-render byte count < first-render byte count AND second-render contains exactly one `\x1b[<n>A` cursor-up.
- `TestRender_ToolResultFrame_NoChange_EmitsNothing` — render same frame twice; assert second-render captured zero bytes after the diff returns empty dirty set.
- `TestRender_FactoryTruthTable_AllNineRows`.
- `TestRender_HandleGenerateStreamPath_GoesThroughRendererNotFmtPrint` — wires up a CLI instance with a fake LLM provider; runs the streaming code path; asserts the captured stdout matches the renderer's signature byte pattern (NOT the today's "%s " token-with-space pattern).

6.3 Challenge (challenges/p1-f18-no-flicker-rendering/)

Four-phase output skeleton (three always-run + one TTY-gated):

```
=== STREAMING-FANCY (always runs) ===
[PASS] tmpdir created at /tmp/p1f18-XXXX
[PASS] constructed ansiRenderer with bytes.Buffer writer + IsTTY=true
[PASS] fake LLM provider emitted 10 tokens via WriteToken
[PASS] captured 247 bytes (mode=fancy)
[PASS] capture contains 10 \\r\\x1b[K sequences (one per token's in-place
[PASS] capture contains exactly 1 trailing \\n (from Commit)
[PASS] capture contains the concatenated token text "Hello, world! This is

=== STREAMING-PLAIN (always runs) ===
[PASS] tmpdir created at /tmp/p1f18-XXXX
[PASS] constructed plainRenderer with bytes.Buffer writer + IsTTY=false
```

```

[PASS] fake LLM provider emitted 10 tokens via WriteToken
[PASS] captured 89 bytes (mode=plain)
[PASS] capture contains zero \\x1b bytes (no ANSI)
[PASS] capture contains zero \\x0d bytes (no CR)
[PASS] capture's text matches the concatenated token text byte-for-byte

=== DIRTY-REGION-DIFF (always runs) ===
[PASS] rendered initial 5-line frame; captured 245 bytes
[PASS] rendered same frame with line 2 changed; captured 28 bytes
[PASS] second-render byte count (28) < first-render byte count (245)
[PASS] second-render contains exactly 1 \\x1b[<n>A cursor-up sequence
[PASS] second-render contains exactly 1 \\r\\x1b[K clear-line sequence
[PASS] all-lines-changed render emits 5 dirty-line emissions (negative con

=== TTY-FALLBACK (always runs) ===
[PASS] constructed renderer via factory with HELIXCODE_RENDER=auto + IsTTY
[PASS] resolved mode = plain
[PASS] ran streaming + tool-block pipelines under plain mode
[PASS] captured 412 bytes; zero \\x1b bytes; zero \\x0d bytes
[PASS] grep -P '\\x1b|\\x0d' over capture returns no matches

=== REAL-TTY (gated; SKIP-OK on non-TTY) ===
[PASS] term.IsTerminal(stdout.Fd()) = true
[PASS] constructed renderer via factory; mode = fancy
[PASS] streamed 10 tokens to os.Stdout via WriteToken
[PASS] renderer.Close() returned nil; cursor restored

```

(or)

```

[SKIP] SKIP-OK: P1-F18 real-TTY phase requires interactive terminal (CI /
SUMMARY: STREAMING-FANCY=7/7 PASS; STREAMING-PLAIN=7/7 PASS; DIRTY-REGION-

```

The Challenge MUST exit non-zero on any byte-signature mismatch. Absence-of-error is NEVER acceptable. A Challenge that reports PASS without observed positive byte evidence is itself a bluff.

7. Cross-platform

ANSI escapes work natively on Linux/macOS. Windows 10+ (build 14931+) supports ANSI sequences when VT mode is enabled — Go 1.16+ auto-enables `ENABLE_VIRTUAL_TERMINAL_PROCESSING` on `os.Stdout` via the runtime when the binary is built with `-buildmode=exe` (the default). For HelixCode targeting Windows 10+, no extra shim is needed; F18 inherits the auto-enablement.

`golang.org/x/term`'s `IsTerminal` and `GetSize` work on Linux/macOS/Windows. Pure Go (no CGO).

The cross-compile `make prod` target (linux/macos/windows) is exercised in T08. The integration test that asserts ANSI byte sequences is gated on `runtime.GOOS != "windows"` ONLY for the cursor-up sequence (`\033[<n>A`) test, because Windows pre-VT-enable would not interpret it; with VT mode on Windows 10+, it works. SKIP-OK marker
 SKIP-OK: P1-F18 cursor-up byte-match test on Windows requires VT

mode probe (covered by Windows-specific manual test) is added for the rare case where the test runner is on a pre-Windows-10 host (vanishing population).

8. Out of scope (deferred)

- **Full screen-mode TUI** (alternative-screen-buffer `\033[?1049h / \033[?1049l`) — F18.5. v1 stays in the normal scrollback; in-place updates happen *within* the visible region.
- **tcell / termbox / bubbletea adoption in the CLI** — F18.5 candidate. v1 deliberately uses pure stdlib + `golang.org/x/term`.
- **SIGWINCH responsiveness** — F18.5. v1 reads width once at factory time.
- **Mouse input** — F18.5.
- **Bracketed paste** — F18.5.
- **Truecolor / 24-bit color negotiation** — F18.5. v1 emits no SGR sequences itself; tool output's embedded SGR is passed through.
- **Stream multiplexer** for interleaved-stream rendering — F18.5 (see §5.3).
- **Terminal-width-aware wrap / truncate** — F18.5 (see §5.5).
- **ANSI-strip option for plain mode tool output** — F18.5 (see §5.4).
- **Slash command** for in-process mode toggling (`/render plain|fancy`) — F18.5 if user demand emerges.

9. Constitutional compliance

- **§11.9 / CONST-035** — Challenge has FOUR always-run phases + one TTY-gated phase. Every phase records positive byte evidence (specific byte sequences, byte counts, signature presence). Mismatch is a hard failure. The four “what counts as no-flicker” criteria (§5.2) each map to a unit + integration + Challenge assertion.
- **CONST-039** — Challenge at `challenges/p1-f18-no-flicker-rendering/` + evidence harness at `tests/integration/cmd/p1f18_challenge/main.go`. Every phase asserts byte content with positive evidence.
- **CONST-042 (No-Secret-Leak)** — The renderer NEVER logs token text or frame content at any level. A unit test scans the renderer's source files for any `logger.Info(*token / logger.Info(*frame / logger.Debug(*token` and FAILS on any match. Rendered output to stdout is by design (the user invoked a streaming prompt; they see the tokens) and not a CONST-042 concern (terminal output is local-only). The Challenge's saved evidence file records byte lengths + signature presence, never the rendered text itself.
- **CONST-043 (No-Force-Push)** — close-out task pushes to all four remotes non-force; explicit user authorization is requested at T10 before pushing.
- **No-Mocks-In-Production (Universal Rule 2)** — the renderer's only test seam is `Options.Writer` (an `io.Writer` for capture) + `Options.IsTTY` (an injectable bool for TTY-detection bypass). Both are constructor-injection seams, not mocks. Production code uses `os.Stdout` and `term.IsTerminal`. No filesystem abstraction; no mocked terminal.

10. Open questions resolved

Q	Answer	Resolution
Q1: renderer technology	(A) custom ANSI/CR	No new deps; <code>\r\033[K</code> for in-place updates; ANSI cursor positioning for multi-line; double-buffered viewport for tool output

Q	Answer	Resolution
Q2: instrumentation surface	(B) streaming tokens + tool result blocks	Streaming LLM tokens flow through <code>WriteToken</code> ; tool-result blocks render via <code>RenderFrame</code> with dirty-region diff; other CLI output (banners, /commands, cobra) stays line-based
Q3: redraw strategy	(B) dirty-region diff	<code>Viewport.Diff</code> returns indices of changed lines; the renderer emits cursor-positioning + clear-line + write ONLY for dirty indices; minimises terminal jitter
Q4: non-TTY graceful degrade	(B) detect via <code>term.IsTerminal</code>	<code>plainRenderer</code> emits zero ANSI / zero <code>\r</code> ; output is byte-identical to <code>fmt.Fprintln</code> line-by-line; complete + correct + grep-safe
Q5: user surface	(B) env var only	<code>HELIXCODE_RENDER=plain\ fancy\ auto</code> (default <code>auto</code>); read once at startup; NO slash; NO cobra

11. Non-obvious decisions (recorded for plan-time review)

1. **fancy on a non-TTY is allowed (user opt-in)** — when `HELIXCODE_RENDER=fancy` is explicitly set and `stdout` is a pipe/file, the user gets raw ANSI in their captured output. Reason: don't second-guess the user's explicit request; the variable is a power-user knob. The default (`auto`) protects the 99% case.
2. **`plainRenderer` strips `\r` from tokens silently** — a token containing `\r` (e.g., the LLM emits "step 1\rstep 2") would corrupt log capture. Reason: log-grep-safety is the entire point of plain mode; preserving the `\r` would defeat the fallback. Documented; no toggle.
3. **`plainRenderer` passes ANSI through verbatim** — but strips `\r`. This asymmetry is deliberate: `\r` corrupts log capture (unrecoverable; mangles `grep`); ANSI in a log file is recoverable (visible noise; some terminal-recasting tools render it; `sed -e 's/\x1b\[ [0-9;]*m//g'` strips it post-hoc). `v1` picks the asymmetry; `v2` may add a strip-ANSI toggle.
4. **`Begin` is auto-called by the first `WriteToken` if not invoked first** — pragmatic for streaming hot paths where `Begin` would be one extra call site. The renderer creates the block state lazily.
5. **`Commit` is the renderer's only "flush" surface** — there's no public `Flush` method. Reason: `Commit` already implies "this block is done"; a separate `Flush` would invite "flush the block but keep accepting writes" which is incompatible with the dirty-region viewport state.
6. **Width is read once at factory time, never re-probed** — Reason: `term.GetSize` is a syscall; doing it per-render is wasteful. `SIGWINCH` handling is explicitly deferred to `F18.5`. Users with mid-process resizes restart the process; `v1` documents the limitation.
7. **No filesystem abstraction; pure `os.Stdout` + `golang.org/x/term`** — the renderer takes an `io.Writer` (constructor seam for tests) and reads `os.Getenv` directly via the factory. No `Filesystem` interface; no `Env` provider. Reason: matches `F17`'s discipline

- (one seam, not five); tests use `bytes.Buffer` for the writer and an injectable `Env` func via `Options.Env`.
8. **The renderer's source files have NO `logger.Info / logger.Debug` of token or frame text** — enforced by a source-scan test (§6.1 Anti-bluff). Reason: `CONST-042` is *easy* to comply with at the renderer (just don't log content); a single accidental `DEBUG` log of “rendered token: %s” in a future patch would re-leak prompts/secrets. The source-scan test pins it.
 9. **No new external deps; `golang.org/x/term` promoted from indirect to direct** — `go.mod` will show one promotion (the `// indirect` marker drops); `go.sum` is unchanged. Reason: `golang.org/x/term` is the `stdlib`-equivalent for terminal probes; pulling in `tcell` or `bubbletea` for `IsTerminal` would be overkill. `go mod tidy` after T02 must produce zero new `go.sum` entries.
 10. **Single-writer concurrency model** (§5.3) — the agent loop is the writer; the renderer's mutex is for safety, not for promised interleaving. Two goroutines streaming into different blocks at once is undefined-behaviour-but-not-crashy in v1. F18.5 may add stream multiplexing if user demand emerges.
 11. **`RenderToolResult` is a renderer-package helper, not registry-side** — Reason: keeps the registry decoupled from the renderer; the helper is called from the CLI's tool-result-printing call sites in `cmd/cli/main.go`. Alternative (registry-side hook) would force every `Tool.Execute` consumer to know about the renderer. The helper is opt-in: callers that don't use it print results the old way (still works).